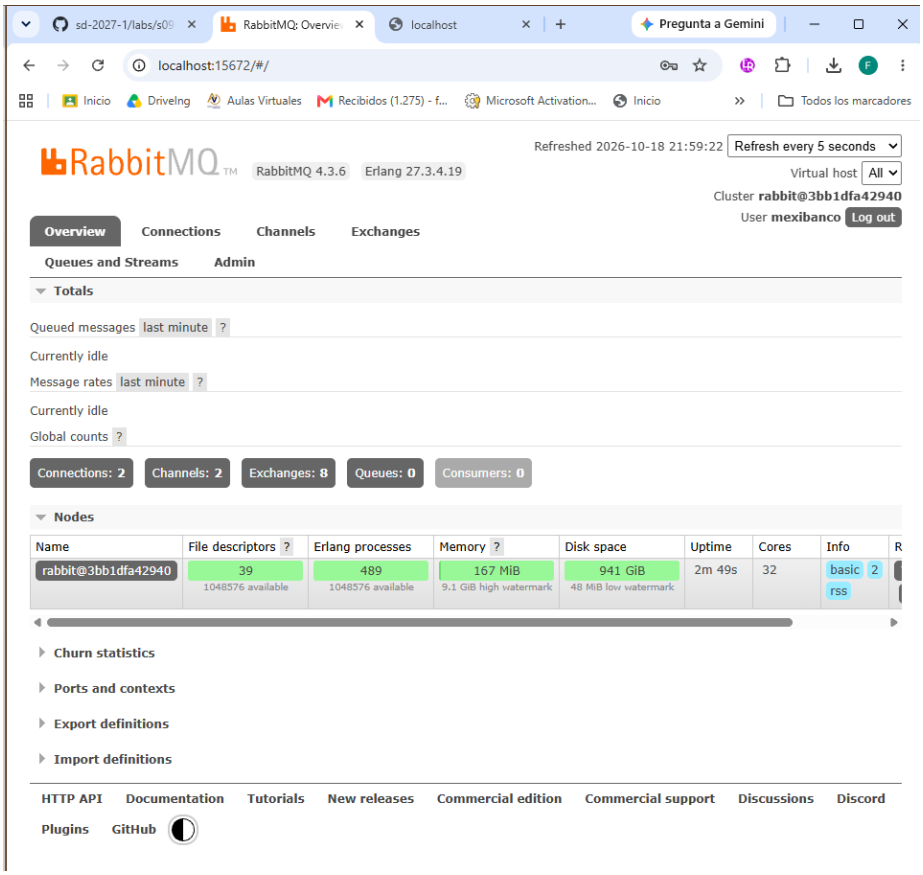
Bitácora · Laboratorio S09

Francisco Javier Pérez Nava

Preparación del entorno

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ docker compose ps
NAME                              IMAGE                                                                                  COMMAN
D                    SERVICE         CREATED              STATUS                     PORTS                                    
mexi-banco-v06-cuenta-1           mexi-banco-v06-cuenta                                                                  "java
-jar app.jar"        cuenta          About a minute ago   Up 47 seconds (healthy)    8080/tcp
mexi-banco-v06-cuenta-2           mexi-banco-v06-cuenta                                                                  "java
-jar app.jar"        cuenta          About a minute ago   Up 48 seconds (healthy)    8080/tcp
mexi-banco-v06-cuenta-3           mexi-banco-v06-cuenta                                                                  "java
-jar app.jar"        cuenta          About a minute ago   Up 47 seconds (healthy)    8080/tcp
mexi-banco-v06-db-1               postgres:17-alpine                                                                     "docke
r-entrypoint.s…"     db              About a minute ago   Up 53 seconds (healthy)    5432/tcp
mexi-banco-v06-discovery-1        sha256:7811c3add1c4599494b546aacf02b709b2aadf6c1b477c47a786599a9d24f49b                 "java
-jar app.jar"        discovery       22 hours ago         Up 7 hours (healthy)       0.0.0.0:8761->8761/tcp, [::]:87
61->8761/tcp
mexi-banco-v06-gateway-1          mexi-banco-v06-gateway                                                                 "java
-jar app.jar"        gateway         About a minute ago   Up 53 seconds (healthy)    0.0.0.0:18080->8080/tcp, [::]:1
8080->8080/tcp
mexi-banco-v06-movimiento-1       mexi-banco-v06-movimiento                                                              "java
-jar app.jar"        movimiento      About a minute ago   Up 48 seconds (healthy)    8080/tcp
mexi-banco-v06-notificacion-1     mexi-banco-v06-notificacion                                                            "java
-jar app.jar"        notificacion    About a minute ago   Up 43 seconds (healthy)    8080/tcp
mexi-banco-v06-rabbitmq-1         rabbitmq:4-management                                                                  "docke
r-entrypoint.s…"     rabbitmq        About a minute ago   Up 53 seconds (healthy)    0.0.0.0:5672->5672/tcp, [::]:56
72->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
mexi-banco-v06-spei-1             mexi-banco-v06-spei                                                                    "java
-jar app.jar"        spei            About a minute ago   Up 48 seconds (healthy)    8080/tcp
mexi-banco-v06-transferencia-1    mexi-banco-v06-transferencia                                                           "java
-jar app.jar"        transferencia   About a minute ago   Up 43 seconds (healthy)    8080/tcp
```



Parte 0. Problema de la notificación síncrona

Parte 0. El problema

Primero apagué el servicio de notificaciones e hice una transferencia de $150 entre dos cuentas, la cuenta de origen tenía $1,000 y la de destino $500, al realizarla recibí un error HTTP 500, pero cuando consulté la transferencia número 5 observé que había quedado en estado ABONADA.

Después revisé los saldos y comprobé que la cuenta de origen había bajado a $850 y la de destino había aumentado a $650, por lo que el dinero sí se había transferido aunque el sistema mostrara un error.

Esto pasó porque el servicio de transferencias todavía dependía de una llamada HTTP a notificaciones y, como estaba apagado, no pudo terminar la operación correctamente aunque el dinero ya se había movido.

Servicio de notificación apagado

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ docker compose stop notificacion
[+] stop 1/1
 ✔ Container mexi-banco-v06-notificacion-1 Stopped                                                    3.4s

franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ docker compose ps -a notificacion
NAME                          IMAGE                         COMMAND               SERVICE        CREATED
  STATUS                      PORTS
mexi-banco-v06-notificacion-1   mexi-banco-v06-notificacion   "java -jar app.jar"   notificacion   4 minutes ago
  Exited (137) 17 seconds ago
```

Estado y saldos tras la falla

Saldos antes

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -i -X POST http://localhost:18080/api/cuentas \
  -H "Content-Type: application/json" \
  -d '{"clabe":"002180000000000001","titular":"Francisco","saldoInicial":1000}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 1e3df5fd7409
Content-Type: application/json
Date: Mon, 19 Oct 2026 05:17:56 GMT

{"id":6,"clabe":"002180000000000001","titular":"Francisco","saldo":1000}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -i -X POST http://localhost:18080/api/cuentas \
  -H "Content-Type: application/json" \
  -d '{"clabe":"002180000000000002","titular":"Cuenta destino","saldoInicial":500}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 46616e686630
Content-Type: application/json
Date: Mon, 19 Oct 2026 05:18:05 GMT

{"id":7,"clabe":"002180000000000002","titular":"Cuenta destino","saldo":500}
```

Estado

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -i http://localhost:18080/api/transferencias/5
HTTP/1.1 200 OK
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 05:23:55 GMT

{"id":5,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":150.00,"fecha":"2026-10-19
T05:20:12.842704Z","estado":"ABONADA","motivo":null,"saldoOrigenTrasCargo":850.00,"intentosDeCompensacion":0,"act
ualizada":"2026-10-19T05:20:12.953300Z"}
```

Saldos tras la falla

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -s http://localhost:18080/api/cuentas/002180000000000001
{"id":6,"clabe":"002180000000000001","titular":"Francisco","saldo":850.00}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -s http://localhost:18080/api/cuentas/002180000000000002
{"id":7,"clabe":"002180000000000002","titular":"Cuenta destino","saldo":650.00}
```

Parte 1. Publicación de eventos

Configuración de RabbitMQ

Primero configuré el exchange transferencias utilizando el método prepararExchange(), dentro de este método utilicé rabbitTemplate.execute() para declarar un exchange de tipo topic y durable.

Elegí esta configuración porque permite enviar los eventos utilizando distintas routing keys, como transferencia.realizada y transferencia.revertida, mientras que la opción durable permite que la definición del exchange se conserve cuando RabbitMQ se reinicia

```java
// NUEVO S09: crear el exchange al iniciar el servicio.
@PostConstruct
public void prepararExchange() {
    rabbitTemplate.execute(channel -> {
        channel.exchangeDeclare("transferencias", "topic", true);
        return null;
    });
}
```

Publicación del evento

Después agregué el método publicarEvento(), que se encarga de preparar la información que vamos a enviar a RabbitMQ, para esto utilicé un Map donde guardo los datos de la transferencia, como el identificador del evento, su tipo, el número de transferencia, las cuentas involucradas, el monto, los saldos y la fecha.

Para generar el eventoId utilicé UUID.nameUUIDFromBytes(), tomando como base el identificador de la transferencia y el tipo de evento, esto permite que si vuelvo a publicar el mismo evento se genere el mismo identificador y así el consumidor pueda reconocer que ya lo había recibido.

Después convierto los datos a JSON utilizando ObjectMapper, preparo el mensaje con MessageBuilder y finalmente lo publico mediante rabbitTemplate.send(), indicando el exchange y la routing key que corresponde al evento.

```java
// NUEVO S09: publicar un evento JSON con los datos del contrato.
private void publicarEvento(Transferencia t, String tipo, String clave, BigDecimal saldoDestino) {
    Map<String, Object> evento = new LinkedHashMap<>();
    // NUEVO S09: UUID estable para el mismo tipo de evento y transferencia.
    evento.put("eventoId", UUID.nameUUIDFromBytes(
        ("transferencia-" + t.getId() + "-" + tipo).getBytes(StandardCharsets.UTF_8)
    ).toString());
    evento.put("tipo", tipo);
    evento.put("transferenciaId", t.getId());
    evento.put("clabeOrigen", t.getClaveOrigen());
    evento.put("clabeDestino", t.getClaveDestino());
    evento.put("monto", t.getMonto());
    evento.put("saldoOrigen", t.getSaldoOrigenTrasCargo());
    evento.put("saldoDestino", saldoDestino);
    evento.put("fecha", Instant.now().toString());

    try {
        byte[] json = objectMapper.writeValueAsBytes(evento);
        Message mensaje = MessageBuilder.withBody(json)
            .setContentType("application/json")
            .build();
        rabbitTemplate.send("transferencias", clave, mensaje);
        log.info("NUEVO S09: evento {} publicado para transferencia {}", tipo, t.getId());
    } catch (JsonProcessingException e) {
        throw new IllegalStateException("No se pudo crear el JSON del evento", e);
    }
}
```

Reemplazo de la llamada HTTP

En el método transferir() eliminé las dos llamadas que se hacían directamente al servicio de notificaciones y cambié esa parte para que primero se guarde la transferencia con el estado COMPLETADA y después se publique el evento TransferenciaRealizada.

```
// NUEVO S09: ya no se llama a notificacion por HTTP.
t.marcarCompletada();
t = guardar(t);

// NUEVO S09: publicar el evento despues de guardar COMPLETADA.
publicarEvento(t, "TransferenciaRealizada", "transferencia.realizada", destino.saldo());
return t;
```

Solo publicamos la reversion si la saga quedo COMPENSADA

También modifiqué el método compensar() para publicar un evento cuando una transferencia no se puede realizar y el dinero regresa a la cuenta de origen, para esto agregué una condición que revisa si el estado final es COMPENSADA y solamente entonces publica TransferenciaRevertida.

```
// NUEVO S09: solo publicamos la reversion si la saga quedo COMPENSADA.
if (t.getEstado() == COMPENSADA) {
    publicarEvento(t, "TransferenciaRevertida", "transferencia.revertida", null);
}
return t;
```

¿Qué garantiza el publisher confirm?

El publisher confirm permite que RabbitMQ le indique al productor si aceptó la publicación de un mensaje, pero esto no significa que el consumidor ya lo haya procesado ni que la notificación se encuentre guardada en la base de datos.

Ademas una confirmación no garantiza por sí sola que el mensaje haya llegado a una cola, ya que si no existe un binding adecuado el evento podría no entregarse a ningún consumidor, por eso es importante configurar correctamente tanto el exchange como las colas.

Transferencia sin notificación

Notificación apagada con RabbitMQ funcionando

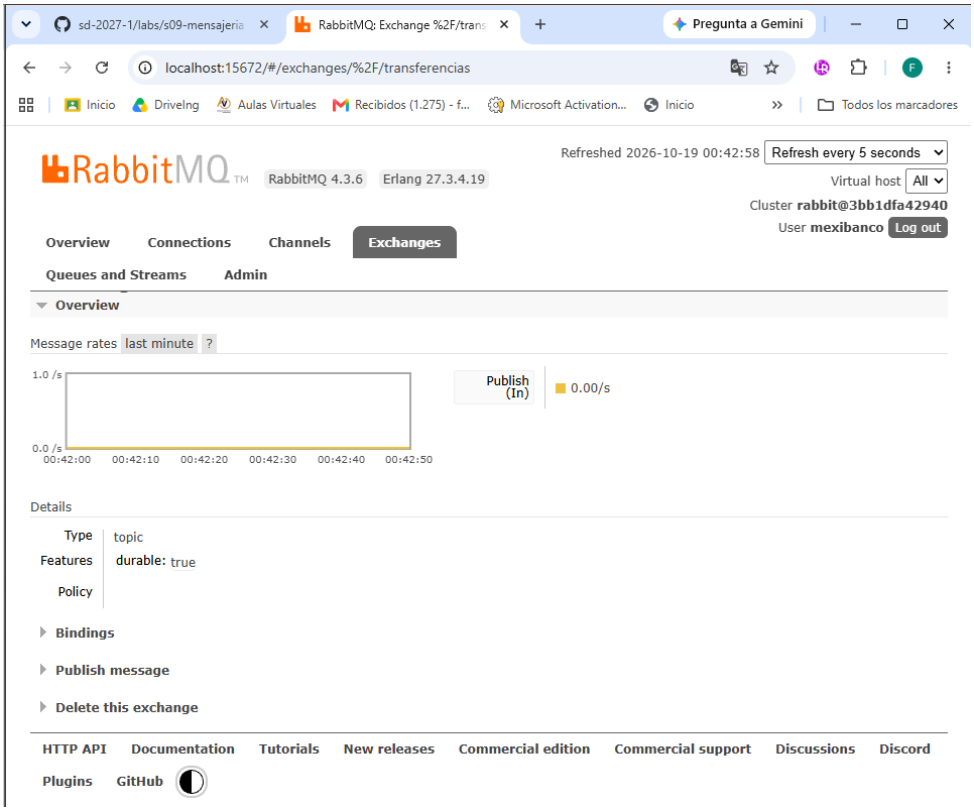
```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ docker compose ps -a
NAME                                IMAGE                                                                                  COMMAN
D                   SERVICE         CREATED         STATUS                       PORTS
mexi-banco-v06-cuenta-1             mexi-banco-v06-cuenta                                                                  "java
-jar app.jar"       cuenta          3 hours ago     Up 3 hours (healthy)         8080/tcp
mexi-banco-v06-cuenta-2             mexi-banco-v06-cuenta                                                                  "java
-jar app.jar"       cuenta          3 hours ago     Up 3 hours (healthy)         8080/tcp
mexi-banco-v06-cuenta-3             mexi-banco-v06-cuenta                                                                  "java
-jar app.jar"       cuenta          3 hours ago     Up 3 hours (healthy)         8080/tcp
mexi-banco-v06-db-1                 postgres:17-alpine                                                                     "docke
r-entrypoint.s…"    db              3 hours ago     Up 3 hours (healthy)         5432/tcp
mexi-banco-v06-discovery-1          sha256:7811c3add1c4599494b546aacf02b709b2aadf6c1b477c47a786599a9d24f49b                 "java
-jar app.jar"       discovery       25 hours ago    Up 10 hours (healthy)        0.0.0.0:8761->8761/tcp, [::]:8761
->8761/tcp
mexi-banco-v06-gateway-1            mexi-banco-v06-gateway                                                                 "java
-jar app.jar"       gateway         3 hours ago     Up 3 hours (healthy)         0.0.0.0:18080->8080/tcp, [::]:180
80->8080/tcp
mexi-banco-v06-movimiento-1         mexi-banco-v06-movimiento                                                              "java
-jar app.jar"       movimiento      3 hours ago     Up 3 hours (healthy)         8080/tcp
mexi-banco-v06-notificacion-1       mexi-banco-v06-notificacion                                                            "java
-jar app.jar"       notificacion    3 hours ago     Exited (137) 5 seconds ago
mexi-banco-v06-rabbitmq-1           rabbitmq:4-management                                                                  "docke
r-entrypoint.s…"    rabbitmq        3 hours ago     Up 3 hours (healthy)         0.0.0.0:5672->5672/tcp, [::]:5672
->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
mexi-banco-v06-spei-1               mexi-banco-v06-spei                                                                    "java
-jar app.jar"       spei            3 hours ago     Up 3 hours (healthy)         8080/tcp
mexi-banco-v06-transferencia-1      mexi-banco-v06-transferencia                                                           "java
-jar app.jar"       transferencia   8 minutes ago   Up 8 minutes (healthy)       8080/tcp
```

Transferencia completada

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -i -X POST http://localhost:18080/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":100}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 06:40:01 GMT

{"id":6,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":100,"fecha":"2026-10-19T06
:40:01.698991392Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":750.00,"intentosDeCompensacion":0,"
actualizada":"2026-10-19T06:40:01.959484043Z"}
```

Verificar el exchange en RabbitMQ



Parte 2. Consumo y desacople en el tiempo

En esta parte trabajé con el archivo ConsumidorDeTransferencias.java, donde agregué lo necesario para que el servicio de notificaciones pudiera recibir los mensajes de RabbitMQ, primero configuré la cola notificacion.transferencias y la conecté con el exchange para que pudiera recibir los eventos de las transferencias, también agregué una cola de errores (DLQ) para guardar los mensajes que no se puedan procesar.

Después utilicé @RabbitListener para que el consumidor esté escuchando los mensajes que llegan, cuando recibe uno primero revisa que tenga la información necesaria y dependiendo del tipo de evento genera las notificaciones, si la transferencia se completó guarda dos avisos, uno para la cuenta de origen y otro para la de destino, pero si fue revertida solamente guarda un aviso indicando que el dinero fue devuelto.

También agregué una tabla para guardar los eventoId de los mensajes que ya se procesaron, esto sirve para que si RabbitMQ manda dos veces el mismo evento no se vuelvan a generar las notificaciones, además hice que el identificador y los avisos se guarden en una misma transacción para evitar problemas si ocurre alguna falla. Por último configuré el ACK manual para que RabbitMQ reciba la confirmación solamente después de guardar los avisos, y en caso de que llegue un mensaje incorrecto se rechaza y se manda a la DLQ, de esta

manera podemos recibir las notificaciones aunque el servicio haya estado apagado, evitar avisos repetidos y manejar los mensajes que tengan errores.

```yaml
rabbitmq:
  host: ${RABBIT_HOST:localhost}
  port: ${RABBIT_PORT:5672}
  username: ${RABBIT_USER:mexibanco}
  password: ${RABBIT_PASSWORD:mexibanco}
  # NUEVO S09: confirmar los mensajes manualmente despues de guardarlos.
  listener:
    simple:
      acknowledge-mode: manual
      prefetch: 1
```

```java
package mx.mexibanco.notificacion.mensajeria;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.rabbitmq.client.Channel;
import jakarta.annotation.PostConstruct;
import mx.mexibanco.notificacion.AvisoDeMovimiento;
import mx.mexibanco.notificacion.NotificacionService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.core.*;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.context.annotation.Bean;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.support.TransactionTemplate;

import java.math.BigDecimal;

/**
 * LAB S09: el consumidor de los eventos de transferencia. Hoy esta vacio a proposito:
notificacion
 * todavia recibe los avisos por HTTP (POST /notificaciones, ver
NotificacionController) y esta clase
 * no es un bean de Spring, asi que no hace nada.
 *
 * Lo que va aqui (el enunciado del lab tiene los nombres exactos):
 *  - declarar la cola notificacion.transferencias, enlazada al exchange
transferencias con el
 *    patron transferencia.*, y su dead letter exchange y dead letter queue;
 *  - un metodo con @RabbitListener que lea cada evento, guarde los avisos y SOLO
DESPUES haga ack;
 *  - que recibir dos veces el mismo evento no deje dos juegos de avisos;
```

```
 *  - que un mensaje que no se puede leer termine en la DLQ en vez de atorar la cola.
 */

// NUEVO S09: Spring reconoce esta clase como consumidor.
@Component
public class ConsumidorDeTransferencias {

    // LAB S09: tu consumidor va aqui.

    // NUEVO S09: herramientas para recibir y guardar mensajes.
    private static final Logger log =
            LoggerFactory.getLogger(ConsumidorDeTransferencias.class);

    private final NotificacionService notificaciones;
    private final JdbcTemplate jdbc;
    private final ObjectMapper json;
    private final TransactionTemplate transaccion;

    public ConsumidorDeTransferencias(
            NotificacionService notificaciones,
            JdbcTemplate jdbc,
            ObjectMapper json,
            PlatformTransactionManager administrador) {

        this.notificaciones = notificaciones;
        this.jdbc = jdbc;
        this.json = json;
        this.transaccion = new TransactionTemplate(administrador);
    }

    // NUEVO S09: declarar el exchange de transferencias.
    @Bean
    public TopicExchange exchangeTransferencias() {
        return new TopicExchange("transferencias", true, false);
    }

    // NUEVO S09: declarar el exchange para mensajes incorrectos.
    @Bean
    public DirectExchange exchangeErrores() {
        return new DirectExchange("transferencias.dlx", true, false);
    }

    // NUEVO S09: cola principal con configuracion de DLQ.
    @Bean
    public Queue colaTransferencias() {
        return QueueBuilder.durable("notificacion.transferencias")
                .deadLetterExchange("transferencias.dlx")
                .deadLetterRoutingKey("notificacion.transferencias")
```

```
            .build();
    }

    // NUEVO S09: conectar la cola principal al exchange.
    @Bean
    public Binding enlaceTransferencias() {
        return BindingBuilder.bind(colaTransferencias())
                .to(exchangeTransferencias())
                .with("transferencia.*");
    }

    // NUEVO S09: cola para mensajes que no se pueden procesar.
    @Bean
    public Queue colaErrores() {
        return QueueBuilder.durable("notificacion.transferencias.dlq")
                .build();
    }

    // NUEVO S09: conectar la DLQ al exchange de errores.
    @Bean
    public Binding enlaceErrores() {
        return BindingBuilder.bind(colaErrores())
                .to(exchangeErrores())
                .with("notificacion.transferencias");
    }

    // NUEVO S09: crear tabla con eventoId unico.
    @PostConstruct
    public void prepararTabla() {
        jdbc.execute("""
                CREATE TABLE IF NOT EXISTS evento_procesado (
                    evento_id VARCHAR(36) PRIMARY KEY
                )
                """);
    }

    // NUEVO S09: recibir eventos de RabbitMQ.
    @RabbitListener(queues = "notificacion.transferencias")
    public void recibir(Message mensaje, Channel canal) throws Exception {

        long etiqueta = mensaje.getMessageProperties().getDeliveryTag();

        // NUEVO S09: primero comprobar que el evento sea valido.
        JsonNode evento;

        try {
            evento = json.readTree(mensaje.getBody());
```

```
            if (evento == null || !evento.isObject()) {
                throw new IllegalArgumentException("JSON invalido");
            }

            String tipo = texto(evento, "tipo");
            String eventoId = texto(evento, "eventoId");

            java.util.UUID.fromString(eventoId);

            if (!tipo.equals("TransferenciaRealizada") &&
                !tipo.equals("TransferenciaRevertida")) {
                throw new IllegalArgumentException("Tipo no valido");
            }

            texto(evento, "clabeOrigen");
            texto(evento, "clabeDestino");
            texto(evento, "transferenciaId");
            texto(evento, "fecha");
            monto(evento, "monto");

            if (tipo.equals("TransferenciaRealizada")) {
                monto(evento, "saldoOrigen");
                monto(evento, "saldoDestino");
            }

        } catch (JsonProcessingException | IllegalArgumentException e) {

            // NUEVO S09: mensaje incorrecto, enviarlo a la DLQ.
            log.error("NUEVO S09: mensaje invalido: {}", e.getMessage());

            canal.basicNack(etiqueta, false, false);
            return;
        }

        try {

            // NUEVO S09: guardar evento y avisos en una misma transaccion.
            transaccion.execute(estado -> {

                String eventoId = evento.get("eventoId").asText();
                String tipo = evento.get("tipo").asText();

                // NUEVO S09: ON CONFLICT evita procesar duplicados.
                int insertados = jdbc.update(
                        """
                        INSERT INTO evento_procesado (evento_id)
                        VALUES (?)
                        ON CONFLICT (evento_id) DO NOTHING
```

```
                        """,
                        eventoId
                );

                if (insertados == 0) {
                    log.info("NUEVO S09: evento duplicado ignorado: {}", eventoId);
                    return null;
                }

                String origen = evento.get("clabeOrigen").asText();
                String destino = evento.get("clabeDestino").asText();
                BigDecimal cantidad = monto(evento, "monto");

                if (tipo.equals("TransferenciaRealizada")) {

                    // NUEVO S09: aviso para la cuenta de origen.
                    notificaciones.notificar(new AvisoDeMovimiento(
                            origen,
                            "TRANSFERENCIA_ENVIADA",
                            cantidad.negate(),
                            monto(evento, "saldoOrigen")
                    ));

                    // NUEVO S09: aviso para la cuenta de destino.
                    notificaciones.notificar(new AvisoDeMovimiento(
                            destino,
                            "TRANSFERENCIA_RECIBIDA",
                            cantidad,
                            monto(evento, "saldoDestino")
                    ));

                } else {

                    // NUEVO S09: aviso cuando se devuelve el dinero.
                    notificaciones.notificar(new AvisoDeMovimiento(
                            origen,
                            "TRANSFERENCIA_REVERTIDA: tu transferencia no se realizo y
te devolvimos el dinero",
                            cantidad,
                            BigDecimal.ZERO
                    ));
                }

                log.info("NUEVO S09: evento procesado: {}", eventoId);
                return null;
            });

            // NUEVO S09: confirmar SOLO despues de guardar en PostgreSQL.
```

```
            canal.basicAck(etiqueta, false);

        } catch (Exception e) {

            // NUEVO S09: errores temporales se reintentan.
            log.error("NUEVO S09: error al guardar avisos", e);
            canal.basicNack(etiqueta, false, true);
        }
    }

    // NUEVO S09: obtener un campo obligatorio del JSON.
    private String texto(JsonNode evento, String campo) {
        JsonNode valor = evento.path(campo);

        if (valor.isMissingNode() || valor.isNull() ||
                !valor.isValueNode() || valor.asText().isBlank()) {
            throw new IllegalArgumentException("Falta el campo " + campo);
        }

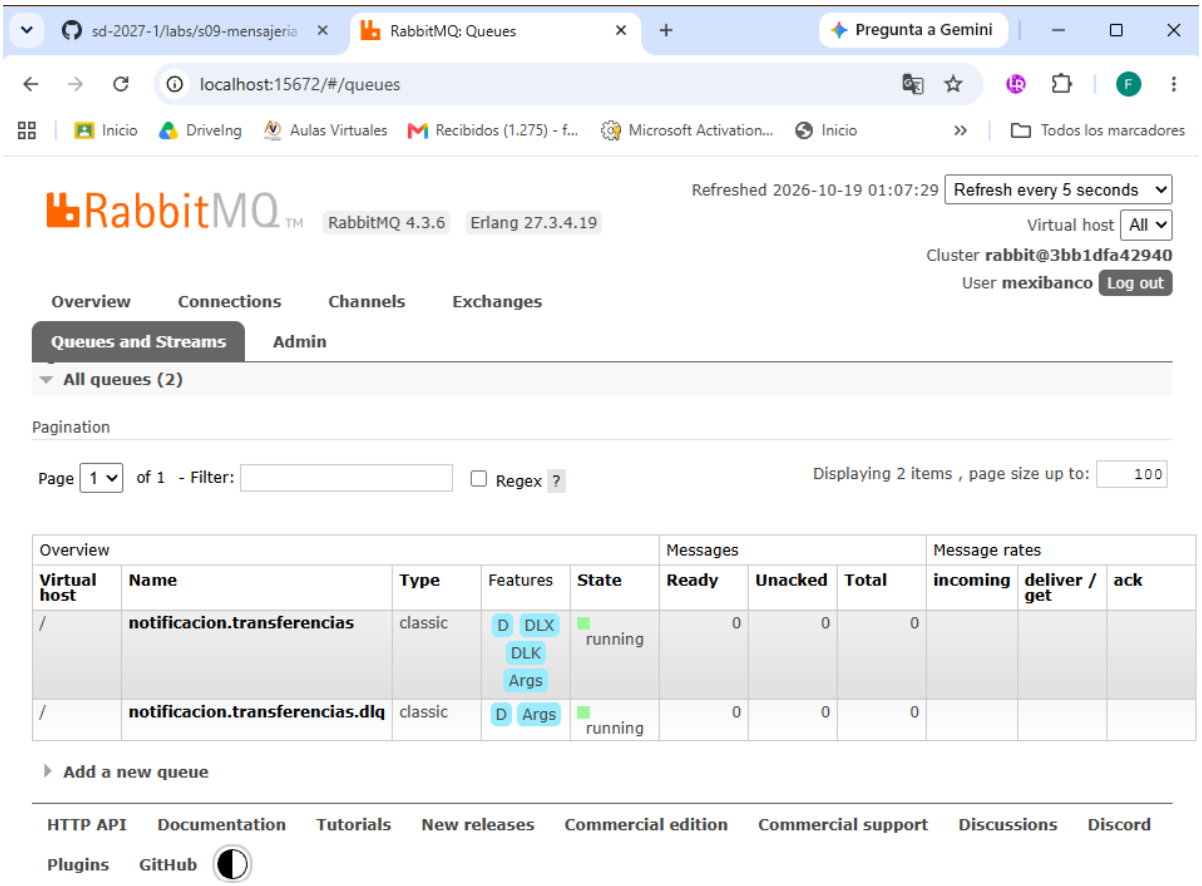
        return valor.asText();
    }

    // NUEVO S09: obtener un monto numerico del JSON.
    private BigDecimal monto(JsonNode evento, String campo) {
        JsonNode valor = evento.path(campo);

        if (!valor.isNumber()) {
            throw new IllegalArgumentException("Monto invalido: " + campo);
        }

        return valor.decimalValue();
    }
}
```

Colas de RabbitMQ



Tres transferencias

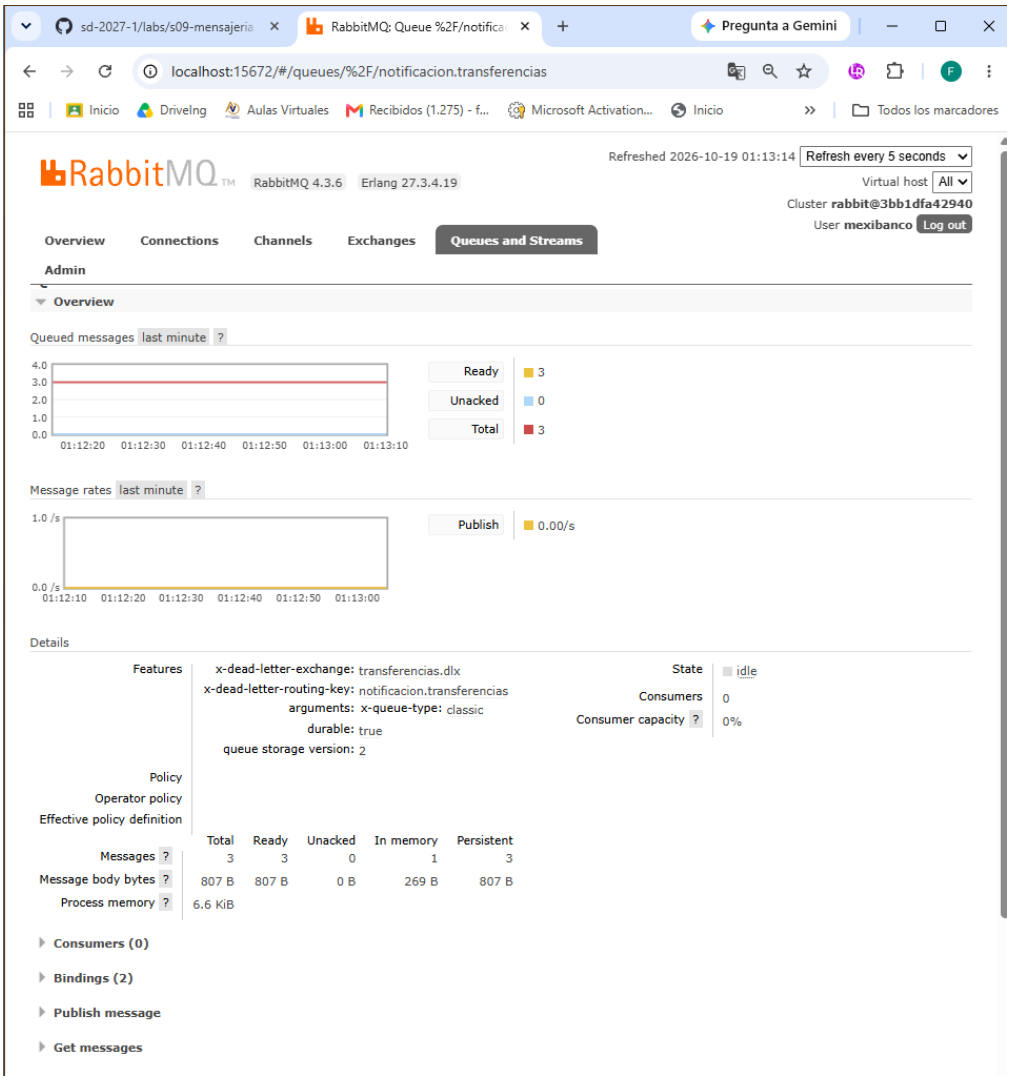
```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -i -X POST http://localhost:18080/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 07:11:49 GMT

{"id":7,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":50,"fecha":"2026-10-19T07:
11:49.413151665Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":700.00,"intentosDeCompensacion":0,"a
ctualizada":"2026-10-19T07:11:49.523348066Z"}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -i -X POST http://localhost:18080/api/transferencias   -H "Content-Type: application/json"   -d '{"claveOr
igen":"002180000000000001","claveDestino":"002180000000000002","monto":60}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 07:11:55 GMT

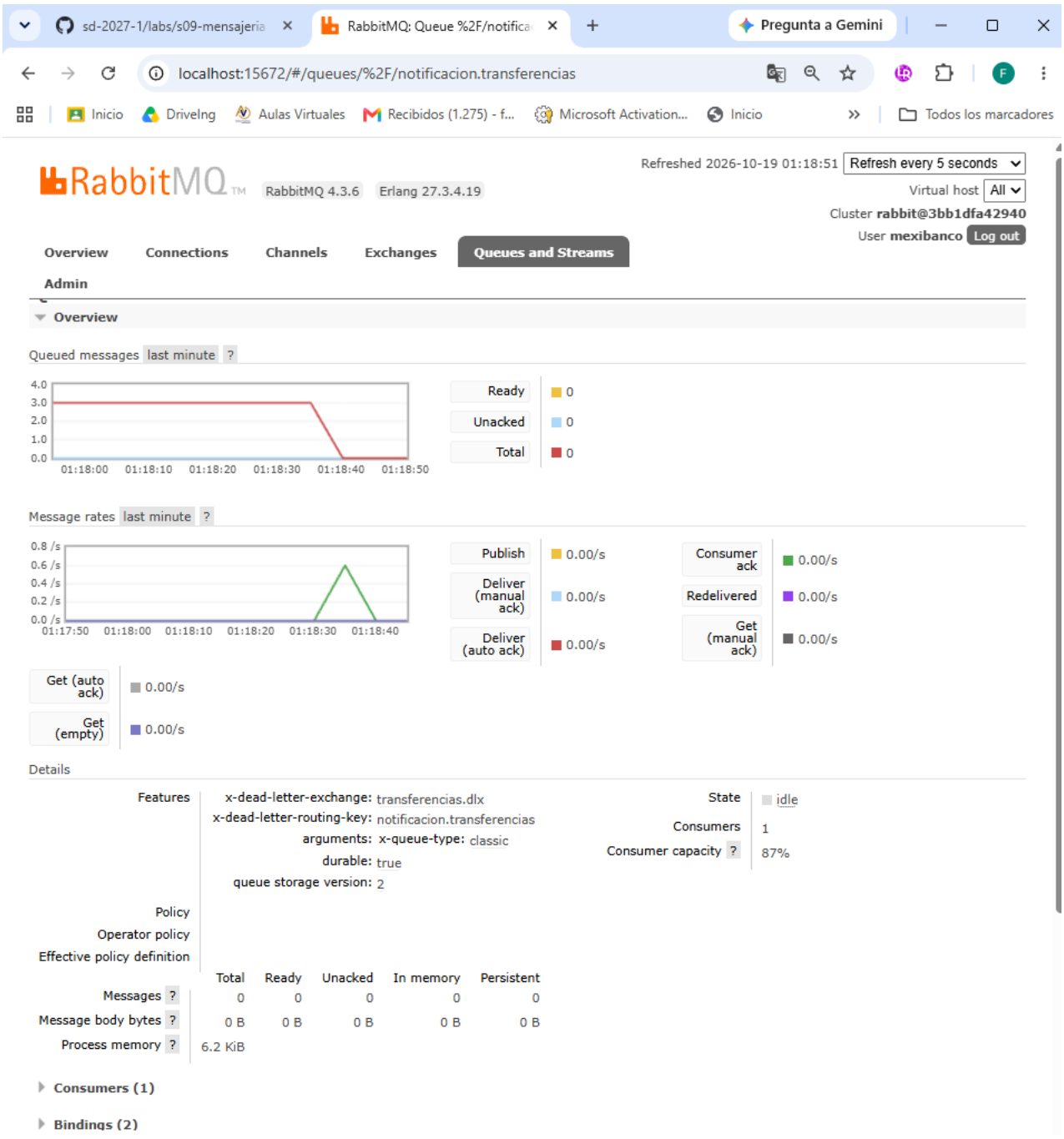
{"id":8,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":60,"fecha":"2026-10-19T07:
11:55.064552909Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":640.00,"intentosDeCompensacion":0,"a
ctualizada":"2026-10-19T07:11:55.186947556Z"}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -i -X POST http://localhost:18080/api/transferencias   -H "Content-Type: application/json"   -d '{"claveOr
igen":"002180000000000001","claveDestino":"002180000000000002","monto":70}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 07:12:00 GMT

{"id":9,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":70,"fecha":"2026-10-19T07:
12:00.256011988Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":570.00,"intentosDeCompensacion":0,"a
ctualizada":"2026-10-19T07:12:00.467818668Z"}
```

Mensajes almacenados en RabbitMQ



Cola vacía después de encender notificación



Bandejas de notificaciones

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -s "http://localhost:18080/api/notificaciones?clabe=002180000000000001"
[{"id":8,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 70, saldo resultante 570.0","fecha":"2026-10-19T07:18:31.376707Z"},{"id":6,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 60, saldo resultante 640.0","fecha":"2026-10-19T07:18:31.342242Z"},{"id":4,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 50, saldo resultante 700.0","fecha":"2026-10-19T07:18:31.182631Z"}]
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -s "http://localhost:18080/api/notificaciones?clabe=002180000000000002"
[{"id":9,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 70, saldo resultante 930.0","fecha":"2026-10-19T07:18:31.379525Z"},{"id":7,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 60, saldo resultante 860.0","fecha":"2026-10-19T07:18:31.346237Z"},{"id":5,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 50, saldo resultante 800.0","fecha":"2026-10-19T07:18:31.313753Z"}]
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
```

Falla de abono configurada

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ docker compose exec transferencia printenv FALLAR_ABONO_A_CLABE
002180000000000002
```

Transferencia compensada

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -i http://localhost:18080/api/transferencias/12
HTTP/1.1 200 OK
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 08:15:43 GMT

{"id":12,"claveOrigen":"00218000000000001","claveDestino":"002180000000000002","monto":10.00,"fecha":"2026-10-19
T08:03:07.831792Z","estado":"COMPENSADA","motivo":"El abono a la CLABE 002180000000000002 fallo: Falla provocada
con FALLAR_ABONO_A_CLABE=002180000000000002","saldoOrigenTrasCargo":540.00,"intentosDeCompensacion":76,"actualiza
da":"2026-10-19T08:15:43.634358Z"}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
```

Aviso de transferencia revertida

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -s "http://localhost:18080/api/notificaciones?clabe=002180000000000001"
[{"id":14,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_REVERTIDA: tu transferencia no se realizo y
 te devolvimos el dinero por 10.0, saldo resultante 0","fecha":"2026-10-19T08:15:43.669913Z"},{"id":12,"clabeCuen
ta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 10, saldo resultante 550.0","fecha":"2026-10-19T07:
57:57.584228Z"},{"id":10,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 10, saldo result
ante 560.0","fecha":"2026-10-19T07:23:31.826850Z"},{"id":8,"clabeCuenta":"002180000000000001","mensaje":"TRANSFER
ENCIA_ENVIADA por 70, saldo resultante 570.0","fecha":"2026-10-19T07:18:31.376707Z"},{"id":6,"clabeCuenta":"00218
0000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 60, saldo resultante 640.0","fecha":"2026-10-19T07:18:31.3422
42Z"},{"id":4,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 50, saldo resultante 700.0"
,"fecha":"2026-10-19T07:18:31.182631Z"}]
```

Parte 3. Duplicado e idempotencia

Transferencia para probar duplicados

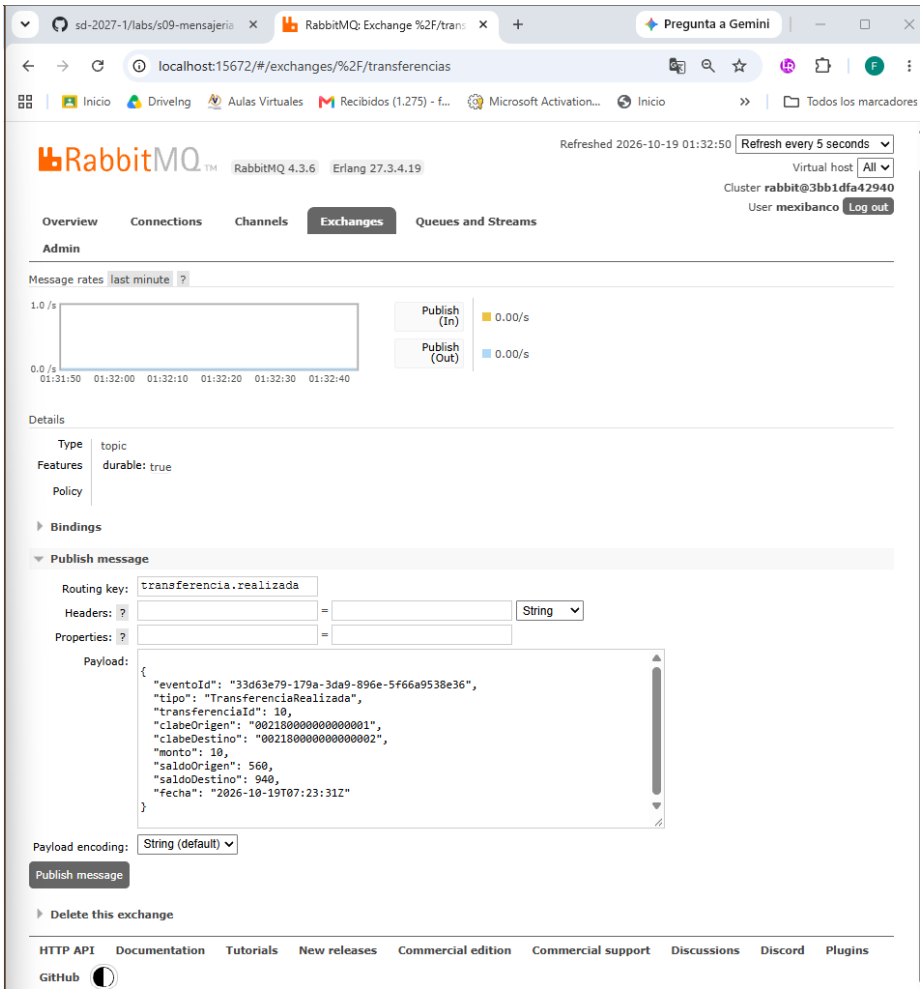
```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -i -X POST http://localhost:18080/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":10}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 07:23:31 GMT

{"id":10,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":10,"fecha":"2026-10-19T07
:23:31.742687850Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":560.00,"intentosDeCompensacion":0,"
actualizada":"2026-10-19T07:23:31.816135269Z"}
```

Avisos antes del duplicado

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -s "http://localhost:18080/api/notificaciones?clabe=002180000000000001"
[{"id":10,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 10, saldo resultante 560.0","fe
cha":"2026-10-19T07:23:31.826850Z"},{"id":8,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA p
or 70, saldo resultante 570.0","fecha":"2026-10-19T07:18:31.376707Z"},{"id":6,"clabeCuenta":"002180000000000001",
"mensaje":"TRANSFERENCIA_ENVIADA por 60, saldo resultante 640.0","fecha":"2026-10-19T07:18:31.342242Z"},{"id":4,"
clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 50, saldo resultante 700.0","fecha":"2026-
10-19T07:18:31.182631Z"}]
```

Publicación del mensaje duplicado

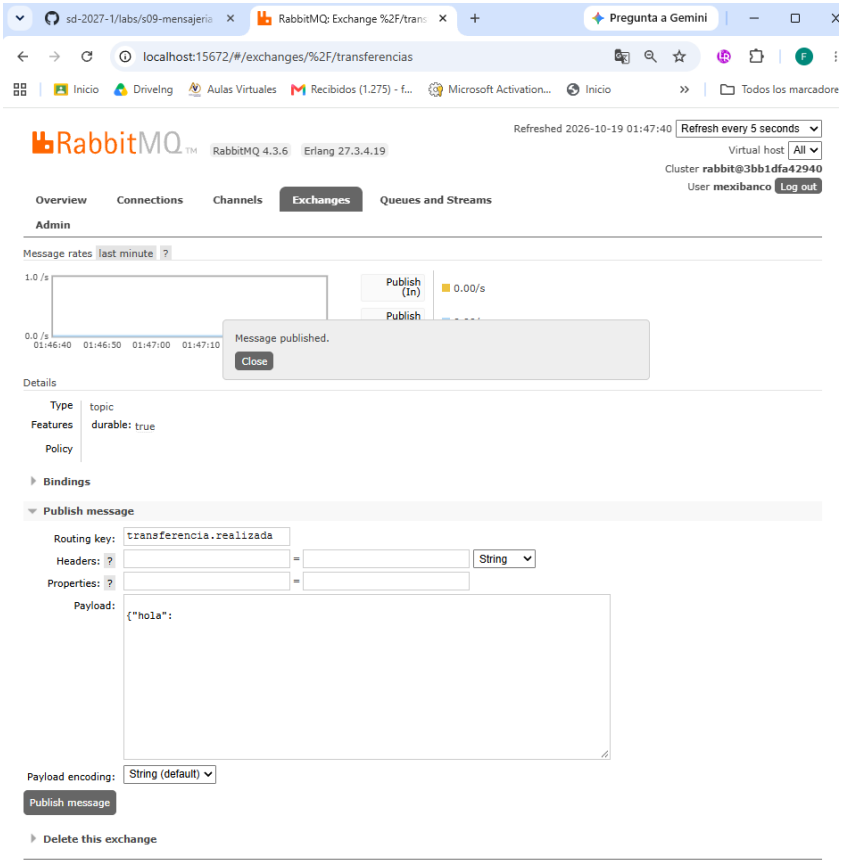


Evento duplicado ignorado

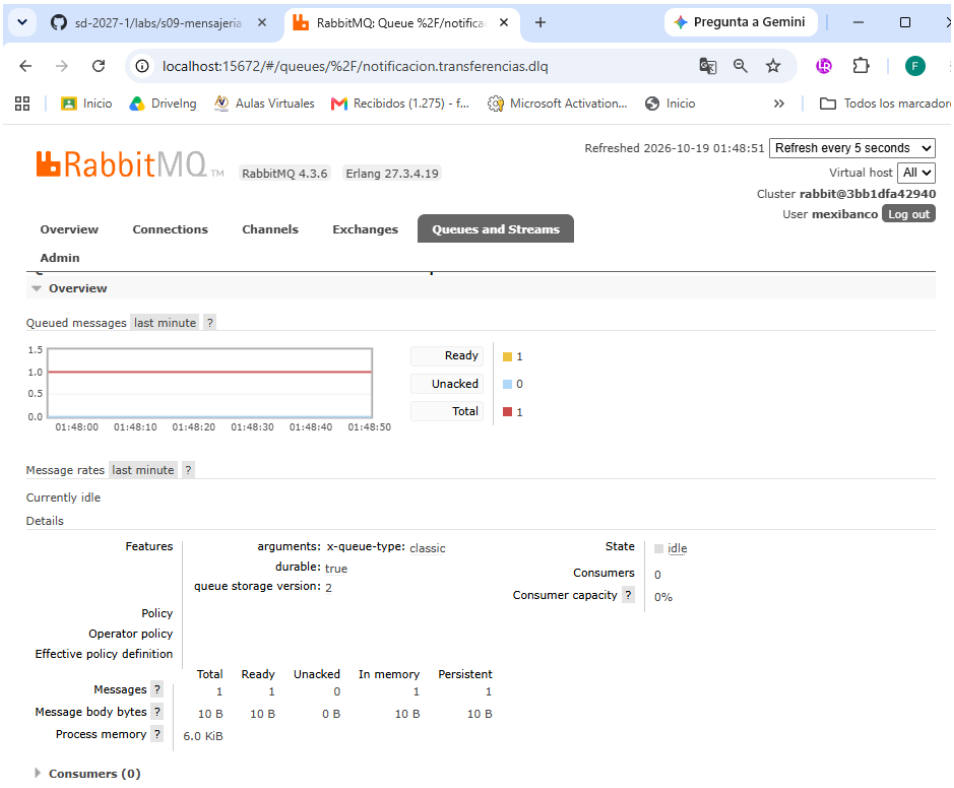
```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -s "http://localhost:18080/api/notificaciones?clabe=002180000000000001"
[{"id":10,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 10, saldo resultante 560.0","fecha":"2026-10-19T07:23:31.826850Z"},{"id":8,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 70, saldo resultante 570.0","fecha":"2026-10-19T07:18:31.376707Z"},{"id":6,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 60, saldo resultante 640.0","fecha":"2026-10-19T07:18:31.342242Z"},{"id":4,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 50, saldo resultante 700.0","fecha":"2026-10-19T07:18:31.182631Z"}]
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ docker compose logs --tail=50 notificacion | grep -i "duplicado"
notificacion-1  | 2026-10-19T07:34:20.108Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : NUEVO S09: evento duplicado ignorado: 33d63e79-179a-3da9-896e-5f66a9538e36
```

Parte 4. Mensaje inválido y DLQ

Mensaje incorrecto publicado



Mensaje en la DLQ



Error detectado por el consumidor

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ docker compose logs --tail=100 notificacion | grep -i "mensaje invalido"
notificacion-1  | 2026-10-19T07:47:39.060Z ERROR 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTrans
ferencias       : NUEVO S09: mensaje invalido: Unexpected end-of-input within/between Object entries
```

Transferencia después del mensaje envenenado

```
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -i -X POST http://localhost:18080/api/transferencias \
  -H "Content-Type: application/json" \
  -d '{"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":10}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Mon, 19 Oct 2026 08:34:42 GMT

{"id":13,"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":10,"fecha":"2026-10-19T08
:34:42.364441529Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":540.00,"intentosDeCompensacion":0,"
actualizada":"2026-10-19T08:34:42.590101254Z"}
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
$ curl -s "http://localhost:18080/api/notificaciones?clabe=0021800000000000001"
[{"id":15,"clabeCuenta":"0021800000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 10, saldo resultante 540.0","fe
cha":"2026-10-19T08:34:42.644626Z"},{"id":14,"clabeCuenta":"0021800000000000001","mensaje":"TRANSFERENCIA_REVERTID
A: tu transferencia no se realizo y te devolvimos el dinero por 10.0, saldo resultante 0","fecha":"2026-10-19T08:
15:43.669913Z"},{"id":12,"clabeCuenta":"0021800000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 10, saldo result
ante 550.0","fecha":"2026-10-19T07:57:57.584228Z"},{"id":10,"clabeCuenta":"0021800000000000001","mensaje":"TRANSFE
RENCIA_ENVIADA por 10, saldo resultante 560.0","fecha":"2026-10-19T07:23:31.826850Z"},{"id":8,"clabeCuenta":"0021
800000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 70, saldo resultante 570.0","fecha":"2026-10-19T07:18:31.376
707Z"},{"id":6,"clabeCuenta":"0021800000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 60, saldo resultante 640.0
","fecha":"2026-10-19T07:18:31.342242Z"},{"id":4,"clabeCuenta":"0021800000000000001","mensaje":"TRANSFERENCIA_ENVI
ADA por 50, saldo resultante 700.0","fecha":"2026-10-19T07:18:31.182631Z"}]
franc@CROSSHAIR MINGW64 ~/Desktop/Sistemas Distribuidos/mexi-banco-s09 (05-mensajeria-base)
```

Preguntas de cierre

1. Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el *publisher confirm*?

Si RabbitMQ está caído, la transferencia puede haber movido el dinero y quedar guardada como COMPLETADA, pero el evento no se podría publicar y la notificación podría perderse, el publisher confirm no soluciona esto porque solamente permite saber si RabbitMQ aceptó el mensaje, pero no lo guarda para enviarlo después cuando el servicio vuelva a funcionar, para solucionar esto necesitaríamos implementar un Outbox.

2. ¿Por qué el ACK va después de guardar y no antes? ¿Qué garantía obtienes con cada orden?

El ACK se hace después de guardar las notificaciones porque si lo hacemos antes y el servicio falla, RabbitMQ pensaría que el mensaje ya fue procesado aunque realmente no se haya guardado nada, lo que podría provocar que se pierda el aviso. En cambio, si primero guardamos y después confirmamos, RabbitMQ puede volver a enviar el mensaje si ocurre alguna falla antes del ACK, pero como utilizamos el eventoId podemos reconocerlo y evitar que se generen notificaciones repetidas

3.Elige y di qué sacrificaste: al quitar la llamada síncrona ganaste que el aviso no tumbe la transferencia. ¿Qué perdiste? Tres líneas: elección, porque, sacrifico.

Elección: cambié la notificación HTTP síncrona por eventos en RabbitMQ.
Porque: el movimiento de dinero ya no tiene que esperar a que notificación esté disponible.
Sacrifico: el aviso puede llegar después y necesito controlar duplicados, fallos de mensajes y publicaciones no confirmadas.